

TITOLO

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione



UNIVERSITÀ POLITECNICA DELLE MARCHE

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

RELATORI:

Emanuele Storti

TESINA DI:

Tarek Naja
Sara Vaccaro

A.A 2024/2025

Indice

INTRODUZIONE	1
Descrizione in linguaggio naturale.	1
GLOSSARIO DEI TERMINI	2
PROGETTAZIONE E SVILUPPO APPLICAZIONE	4
Analisi dei requisiti	4
Requisiti Funzionali	4
Requisiti Non Funzionali: Kotlin (App Nativa)	5
Requisiti Non Funzionali: Flutter (App Cross-Platform)	5
Descrizione dei Casi d'Uso	7
Mockup	12
Verifica e Validazione del Software: Unit Tests	13
Test Manuali	13
Unit Test	13

Introduzione

Descrizione in linguaggio naturale

Il progetto consiste nello sviluppo di **Paws**, un'applicazione mobile pensata per amanti degli animali privati e rifugi. L'obiettivo dell'applicazione è offrire uno strumento per facilitare l'adozione e la condivisione di annunci relativi a cuccioli ed animali domestici, tenendo traccia delle schede di dettaglio dei singoli animali e consentendo il contatto diretto tra gli utenti.

Il sistema prevede due macro-tipologie di utenti:

- **Utenti privati:** individui che utilizzano l'applicazione per consultare gli annunci disponibili, salvare gli animali preferiti nella propria sezione dedicata, pubblicare nuovi annunci di adozione, filtrare la ricerca per razza, età, sesso e tipologia di utente, contattare direttamente i proprietari tramite chiamata telefonica, nonché cercare altri utenti e rifugi per animali (animal shelters) per visionarne i profili e i relativi post.
- **Rifugi e canili (Animal shelters):** strutture ed organizzazioni dedicate che dispongono di funzionalità per la pubblicazione e gestione di annunci di adozione su più ampia scala, promuovendo la visibilità degli animali in cerca di una casa e gestendo il proprio profilo informativo.

Sintesi dell'Approccio

Per la realizzazione del sistema, lo sviluppo dell'applicazione mobile è stato suddiviso in due percorsi complementari:

- **Applicazione Android nativa:** sviluppata in Kotlin sfruttando l'architettura nativa Android e la libreria di componenti UI, garantendo elevate prestazioni, gesture fluide (swipe-to-favorite) ed integrazione sia con i servizi di sistema (come l'avvio delle chiamate) sia con servizi di backend esterni. Nello specifico, sono stati utilizzati Firebase per l'autenticazione e la memorizzazione delle credenziali degli utenti e Supabase per il caricamento delle immagini e l'archiviazione dei dati completi relativi ai cuccioli.
- **Applicazione clone cross-platform:** realizzata in Flutter per consentire la portabilità del codice a partire da un'unica codebase, replicando fedelmente le funzionalità, l'identità visiva e l'interfaccia dell'applicazione nativa.

Questo approccio parallelo consente di analizzare e confrontare i vantaggi dello sviluppo nativo in termini di integrazione con il sistema operativo e risposta delle gesture rispetto ai benefici legati alla velocità di sviluppo e riusabilità offerti dalle moderne tecnologie cross-platform

Glossario dei Termini

Il seguente glossario definisce in modo univoco i termini di business e tecnologici utilizzati nell'ambito del progetto per evitare ambiguità durante le fasi di analisi, progettazione e sviluppo.

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Utente Privato	Persona fisica registrata che utilizza l'app per cercare cuccioli, salvare preferiti e pubblicare annunci di adozione	BUSINESS	Privato, User
Animal Shelter	Struttura d'accoglienza o associazione no-profit che utilizza l'app per cercare cuccioli, salvare preferiti e pubblicare annunci di adozione	BUSINESS	Rifugio, Ente di adozione
Cucciolo	Entità principale dell'applicazione che rappresenta un animale pubblicato con foto, nome, età, sesso, tipo e descrizione	BUSINESS	Animale, Post, Pet, Puppy
Scheda Dettaglio	Schermata informativa dedicata ad un singolo cucciolo contenente tutti i dettagli dell'animale e i contatti del proprietario	BUSINESS	Dettagli Cucciolo, Puppy Details
Preferiti	Sezione dell'applicazione in cui l'utente raccoglie e salva i cuccioli a cui è interessato	BUSINESS	Saved, Mi Piace, Favorites
Notifica	Messaggio automatico inviato al proprietario di un annuncio quando un altro utente aggiunge/rimuove il suo cucciolo ai preferiti	BUSINESS	Avviso, Notification
Filtro di Ricerca	Strumento per restringere l'elenco degli animali visualizzati in base a parametri quali età, sesso, tipo di animale e tipo di utente	BUSINESS	Criterio di Selezione, Search Filter

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Account	Insieme delle credenziali di accesso (email e password) e dei dati personali che identificano un utente nel sistema	TECNICO	Profilo
Swipe to Favorite	Gesto di scorrimento orizzontale della scheda verso destra che consente di salvare un cucciolo nei preferiti	TECNICO	Gesture Swipe, Scorrimento Destra
Firebase	Servizio cloud utilizzato per la gestione della registrazione, dell'autenticazione sicura e della sessione degli utenti	TECNICO	Auth, Gestione Accessi
Cloud Firestore	Database NoSQL in tempo reale utilizzato per memorizzare i dati degli utenti, dei post, dei preferiti e delle notifiche	TECNICO	DB NoSQL, Firestore
Supabase	Servizio di Object Storage utilizzato per l'hosting e l'erogazione performante delle immagini del profilo e delle foto dei cuccioli	TECNICO	Storage Cloud

Progettazione e sviluppo applicazione

Nella presente sezione si analizzano in modo approfondito i requisiti del sistema, procedendo alla loro suddivisione secondo le seguenti categorie:

- **Requisiti funzionali**, che definiscono le specifiche funzionalità che il sistema è tenuto a fornire.
- **Requisiti non funzionali**, che individuano i vincoli e le qualità che il sistema deve soddisfare, quali ad esempio l'usabilità, la sicurezza e le prestazioni.

Analisi dei requisiti

Requisiti Funzionali

Area: Gestione utente

- **RF1: Registrazione utente:** il sistema deve permettere la creazione di un account inserendo email, password, nome, cognome, città, provincia, CAP, numero di telefono, username e la selezione del tipo di account (Utente Privato o Animal Shelter).
- **RF2: Accesso utente:** il sistema deve consentire agli utenti registrati di accedere all'applicazione inserendo le proprie credenziali.
- **RF3: Modifica profilo:** il sistema deve permettere all'utente di modificare i propri dati personali, aggiornare la foto profilo, richiedere il reset della password via email e modificare la tipologia di account.
- **RF4: Logout:** il sistema deve consentire all'utente di effettuare un logout sicuro dalla sessione attiva.
- **RF5: Elimina account:** il sistema deve permettere all'utente di eliminare definitivamente il proprio account.

Area: Gestione post

- **RF6: Creazione cucciolo:** il sistema deve permettere all'utente di caricare un nuovo annuncio specificando nome del cucciolo, tipo di animale (cane, gatto, uccello), sesso (maschio, femmina), età, didascalia descrittiva e foto dell'animale.
- **RF7: Modifica cucciolo:** il sistema deve permettere all'autore dell'annuncio di modificare le informazioni inserite.
- **RF8: Elimina cucciolo:** il sistema deve permettere all'autore dell'annuncio di eliminare definitivamente il post e la relativa immagine memorizzata nel cloud.
- **RF9: Visualizzazione post e scheda dettaglio:** il sistema deve mostrare i post dei cuccioli pubblicati e consentire la visualizzazione delle informazioni complete dell'animale e del proprietario.

Area: Preferiti e interazioni

- **RF10: Aggiunta e rimozione dai preferiti:** il sistema deve permettere di aggiungere un cucciolo ai preferiti e di rimuoverlo premendo sul cuore nella sezione preferiti.
- **RF11: Calcolo dei mi piace:** il sistema deve calcolare e mostrare in tempo reale il numero totale di salvataggi ricevuti per ciascun post.

- **RF12: Chiamata telefonica:** il sistema deve consentire di avviare una chiamata verso il numero di telefono del proprietario.

Area: Ricerca e filtri

- **RF13: Ricerca:** il sistema deve permettere di cercare utenti o cuccioli inserendo il testo nella barra di ricerca.
- **RF14: Filtraggio:** il sistema deve consentire il filtraggio degli annunci in base a età, sesso, tipo di animale e tipo di utente.

Area: Notifiche

- **RF15: Generazione notifica:** il sistema deve generare una notifica destinata al proprietario dell'annuncio quando un altro utente aggiunge/rimuove il suo cucciolo ai/dai preferiti.
- **RF16: Visualizza notifiche:** il sistema deve mostrare all'utente un elenco delle notifiche ricevute.
- **RF17: Elimina notifiche:** il sistema deve permettere all'utente l'eliminazione delle notifiche.

Requisiti Non Funzionali: Kotlin (App Nativa)

- **RNF1-K: Tecnologia di Sviluppo e SDK:** L'applicazione nativa è sviluppata in Kotlin con Android SDK, strutturata mediante Activity e Fragment per la navigazione tra le varie sezioni della schermata principale.
- **RNF2-K: Layout ed Interfaccia Grafica:** L'interfaccia utente è realizzata interamente mediante Layout XML, utilizzando i componenti Material Design e la libreria ViewBinding per il collegamento sicuro tra codice Kotlin e layout.
- **RNF3-K: Autenticazione e Database Cloud:** La gestione della sessione utente e la persistenza dei dati di dominio (Post, Profili Utente, Notifiche, Preferiti) sono gestite tramite Firebase Auth per l'autenticazione ed a Cloud Firestore per la lettura/scrittura in tempo reale dei dati.
- **RNF4-K: Cloud Storage per i Media:** L'upload e la gestione dello storage remoto delle immagini di profili e cuccioli sono realizzati tramite l'integrazione diretta con Supabase Storage.
- **RNF5-K: Rendering e Caching delle Immagini:** Il caricamento asincrono, il rendering ed il caching locale delle immagini da URL remoto nelle ImageView dell'interfaccia utente sono affidati alla libreria Glide.
- **RNF6-K: Visualizzazione Dati e RecyclerView:** L'applicazione organizza la presentazione delle liste e delle griglie di dati (post, cuccioli, notifiche, preferiti) mediante RecyclerView e Adapter dedicati (FeedAdapter, GridPostAdapter, FavoritesAdapter, NotificationAdapter, SearchAdapter).

Requisiti Non Funzionali: Flutter (App Cross-Platform)

- **RNF1-F: Tecnologia di Sviluppo Cross-Platform:** L'applicazione cross-platform è sviluppata in Dart con il framework Flutter (SDK ^3.12.2), garantendo la compatibilità sia per dispositivi Android sia per iOS con un'unica codebase.
- **RNF2-F: Servizi e Modelli Dati:** La gestione dei dati di dominio e della logica di backend è strutturata mediante Service dedicati (FirebaseService, SupabaseService) e Modelli di dati fortemente tipizzati (UserModel, PuppyPostModel, NotificationModel).
- **RNF3-F: Autenticazione e Cloud Firestore:** La gestione dell'autenticazione utente e le operazioni di lettura/scrittura sulle collezioni cloud (Cuccioli, Post, Preferiti, Notifiche, Utenti) sono affidate ai pacchetti ufficiali firebase_auth e cloud_firestore.
- **RNF4-F: Cloud Storage su Supabase:** Il caricamento e l'hosting remoto dei file multimediali (immagini dei cuccioli nel bucket puppy-images ed avatar utenti nel bucket profile-images) sono realizzati tramite il pacchetto supabase_flutter.

- **RNF5-F: Gestione dell'Interfaccia e dello Stato:** L'interfaccia utente e l'aggiornamento dello stato reattivo sono realizzati mediante i componenti nativi `StatefulWidget` e `StatelessWidget` di Flutter, con aggiornamento dinamico gestito tramite `setState()` e l'invocazione diretta dei `Service`.
- **RNF6-F: Design System e Caching Immagini:** L'applicazione impiega la classe centralizzata `AppTheme` (`lib/theme/app_theme.dart`) per la definizione di colori (`AppColors`), stili ed elementi grafici, e la libreria `cached_network_image` per la visualizzazione ottimizzata delle immagini di rete con placeholder di caricamento.

Descrizione dei Casi d'Uso

Di seguito sono descritti nel dettaglio i principali casi d'uso per l'applicazione.

Area: Gestione utente

Caso d'uso: RegistrazioneUtente	
ID	UC-1
Breve descrizione	Un Utente crea un nuovo account per accedere all'applicazione.
Attori primari	Utente non autenticato (Ospite)
Attori secondari	Sistema di Autenticazione (Firebase Auth), Database (Cloud Firestore)
Precondizioni	1. L'applicazione è avviata e l'Utente si trova nella schermata di registrazione.
Sequenza degli eventi principale	1. L'Utente inserisce i propri dati: email, password, nome, cognome, città, provincia, CAP, numero di telefono e username. 2. L'Utente seleziona la tipologia di account (Utente Privato o Animal Shelter). 3. L'Utente convalida la registrazione premendo il pulsante dedicato. 4. Il sistema valida i dati inseriti. 5. Il sistema registra il nuovo account su Firebase Auth e ne salva le informazioni su Cloud Firestore. 6. L'Utente viene reindirizzato alla schermata principale e risulta autenticato.
Postcondizioni	1. L'account viene creato ed è abilitato all'uso, e l'Utente risulta autenticato nel sistema.
Sequenza degli eventi alternativa	1. If i dati non sono validi o l'email risulta già registrata, allora 1.1. Il sistema mostra un messaggio d'errore. 1.2. Il sistema invita l'Utente a correggere i campi errati o a inserire un indirizzo differente.

Caso d'uso: AccessoUtente	
ID	UC-2
Breve descrizione	Un Utente registrato effettua l'accesso all'applicazione inserendo le proprie credenziali.
Attori primari	Utente registrato
Attori secondari	Sistema di Autenticazione (Firebase Auth)
Precondizioni	1. L'Utente possiede un account valido e si trova nella schermata di login.
Sequenza degli eventi principale	1. L'Utente inserisce la propria email e password. 2. L'Utente preme il pulsante di accesso. 3. Il sistema verifica la correttezza delle credenziali. 4. Il sistema autentica l'Utente e lo reindirizza alla schermata principale (Feed).
Postcondizioni	1. L'Utente è autenticato con successo e può accedere alle funzionalità riservate del sistema.
Sequenza degli eventi alternativa	1. If le credenziali inserite risultano errate o inesistenti, allora 1.1. Il sistema mostra un messaggio d'errore per credenziali non valide. 1.2. Il modulo viene pulito o l'Utente è invitato a riprovare l'inserimento.

Caso d'uso: ModificaUtente	
ID	UC-3
Breve descrizione	L'Utente visualizza e aggiorna i propri dati personali o la foto profilo.
Attori primari	Utente autenticato
Attori secondari	Database (Cloud Firestore), Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato e si trova nella sezione di gestione del proprio profilo.
Sequenza degli eventi principale	1. L'Utente seleziona l'opzione di modifica profilo. 2. Il sistema mostra i dati attuali precompilati nei rispettivi campi. 3. L'Utente modifica le informazioni desiderate (es. città, telefono, foto profilo). 4. L'Utente preme il pulsante per salvare le modifiche. 5. Il sistema valida i nuovi dati. 6. Il sistema aggiorna le informazioni nel database (ed eventualmente carica la nuova foto nello storage). 7. Il sistema mostra un messaggio di conferma dell'avvenuto aggiornamento.
Postcondizioni	1. I dati dell'account Utente sono aggiornati correttamente nel sistema.
Sequenza degli eventi alternativa	1. If si verifica un errore di connessione o un dato non rispetta i vincoli di validazione, allora 1.1. Il sistema mostra un messaggio d'errore specifico (tramite Snackbar o Dialog). 1.2. I dati precedenti non vengono sovrascritti.

Caso d'uso: Logout	
ID	UC-4
Breve descrizione	L'Utente termina la propria sessione attiva in modo sicuro.
Attori primari	Utente autenticato
Attori secondari	Sistema di Autenticazione (Firebase Auth)
Precondizioni	1. L'Utente è attualmente autenticato nell'applicazione.
Sequenza degli eventi principale	1. L'Utente naviga nelle impostazioni del profilo e seleziona l'opzione di logout. 2. Il sistema richiede conferma dell'azione tramite una finestra di dialogo. 3. L'Utente conferma la volontà di disconnettersi. 4. Il sistema revoca i token di accesso e distrugge la sessione attiva. 5. L'Utente viene reindirizzato alla schermata di benvenuto o login.
Postcondizioni	1. La sessione è terminata e non è più possibile accedere ai dati protetti senza effettuare un nuovo accesso.
Sequenza degli eventi alternativa	1. If l'Utente annulla l'azione nella finestra di conferma, allora 1.1. L'azione di disconnessione viene annullata e l'Utente rimane autenticato.

Caso d'uso: EliminaUtente	
ID	UC-5
Breve descrizione	L'Utente richiede l'eliminazione definitiva del proprio account e di tutti i dati associati.
Attori primari	Utente autenticato
Attori secondari	Firebase Auth, Database (Cloud Firestore), Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato e si trova nella sezione di gestione account.
Sequenza degli eventi principale	1. L'Utente seleziona l'opzione per eliminare l'account. 2. Il sistema mostra un avviso indicando che l'operazione è irreversibile e richiede un'ulteriore conferma di sicurezza (es. reinserimento della password). 3. L'Utente fornisce la conferma richiesta. 4. Il sistema procede all'eliminazione a cascata: cancella le immagini collegate su Supabase Storage, rimuove i post, i preferiti e il profilo da Cloud Firestore, ed elimina l'account da Firebase Auth. 5. L'Utente viene disconnesso e reindirizzato alla schermata iniziale.
Postcondizioni	1. L'account e tutti i dati ad esso correlati sono permanentemente rimossi dal sistema.
Sequenza degli eventi alternativa	1. If l'Utente annulla l'operazione nella finestra di conferma o fallisce l'autenticazione di sicurezza, allora 1.1. Il sistema interrompe la procedura di eliminazione. 1.2. L'Utente viene riportato alle impostazioni del profilo senza modifiche ai dati.

Mockup

Il mockup dell'applicativo GESTRi è stato realizzato con Figma, uno strumento per la progettazione e la prototipazione di interfacce utente. Ogni mockup rappresenta la versione progettuale dell'interfaccia, definendo struttura, elementi grafici e funzionalità prima della fase di sviluppo.



Figura 1: Mockup login

Verifica e Validazione del Software: Unit Tests

I seguenti test rappresentano l'insieme delle attività di verifica implementate per garantire il corretto funzionamento del progetto. Sono stati adottati due approcci complementari: test automatici lato codice e test manuali delle API.

Test Manuali

Per verificare le API è stata utilizzata l'applicazione Postman, uno strumento che consente di inviare richieste HTTP e analizzare le risposte. Questi test non rientrano nel diagramma degli Unit Test, ma rappresentano un'importante componente di validazione dell'integrazione e del comportamento delle API lato client-server.

Unit Test

La sezione presenta i test unitari eseguiti sul modulo Attività di GESTRi, con l'obiettivo di verificarne la correttezza funzionale e assicurare l'affidabilità del relativo codice.

```
1 class AttivitaIntegrationTests(TestCase):
2     def setUp(self):
3         self.client = Client()
4         # create users
5         self.staff = Utente.objects.create(email='staff@example.com', ruolo=
            Ruolo.STAFF)
6         self.staff.set_password('pass')
7         self.staff.save()
```

Unit Test 1: Attivita